

- Họ tên: Hồ Tuấn Huy
- MSSV: 20225856

Báo cáo kết quả bài tập Sinh mã đích

I. Kết quả thực hiện chương trình với tệp không chứa chương trình con (example1.kpl)

1. Mã nguồn file test

```
tests > example1.kpl
1  PROGRAM CODEGEN9;
2  TYPE T = INTEGER;
3  VAR S : T;
4      I : INTEGER;
5
6  BEGIN
7      S:=0;
8      I:=1;
9      WHILE I<=5 DO
10 BEGIN
11     S:=S+I*I;
12     I:=I+1;
13 END;
14 CALL WRITEI(S);
15 CALL WRITELN;
16 END.
```

2. Mô tả chương trình

CODEGEN9 tính tổng bình phương các số nguyên từ 1 đến 5 và in kết quả:

- Khai báo kiểu T = INTEGER.
- Biến S (tổng) = 0, biến I (đếm) = 1.
- Vòng lặp WHILE I <= 5: mỗi lần cộng I^2 vào S, rồi tăng I lên 1.
- Kết quả: $S = 1^2 + 2^2 + 3^2 + 4^2 + 5^2 = 55$.
- In S bằng WRITEI(S), sau đó WRITELN.

3. Generated code

```
● → Bai5_Type_Code_Gen ./kplc tests/example1.kpl
0: J 1
1: INT 6
2: LA 0,4
3: LC 0
4: ST
5: LA 0,5
6: LC 1
7: ST
8: LV 0,5
9: LC 5
10: LE
11: FJ 25
12: LA 0,4
13: LV 0,4
14: LV 0,5
15: LV 0,5
16: ML
17: AD
18: ST
19: LA 0,5
20: LV 0,5
21: LC 1
22: AD
23: ST
24: J 8
25: LV 0,4
26: WRI
27: WLN
28: HL
○ → Bai5_Type_Code_Gen █
```

4. Giải thích chi tiết

- **Cấu trúc của stack frame:**
 - Offset = 0: RV (Return Value)
 - Offset = 1: DL (Dynamic Link)
 - Offset = 2: RA (Return Address)
 - Offset = 3: SL (Static Link)
 - Offset = 4: S (Biến S)
 - Offset = 5: I (Biến I)
- **Khởi tạo (0–1):**
 - J 1: Nhảy đến lệnh 1 (bỏ qua khai báo).
 - INT 6: Cấp phát 6 ô nhớ cho stack frame.
- **Gán giá trị ban đầu (2–7):**
 - LA 0,4; LC 0; ST: S := 0 (load địa chỉ S, load 0, store)
 - LA 0,5; LC 1; ST: I := 1 (load địa chỉ I, load 1, store)
- **Vòng lặp WHILE (8–24):**
 - *Kiểm tra điều kiện (8–11):*
 - LV 0,5; LC 5: Load I và 5

- LE: So sánh $\leq$
- FJ 25: Nếu sai, nhảy đến 25 (thoát)
- **Thân vòng lặp (12–23):**
 - $S := S + I * I$ (12–18):
 - LA 0,4: Địa chỉ S
 - LV 0,4: Giá trị S
 - LV 0,5; LV 0,5: Load I hai lần
 - ML: Nhân $I * I$
 - AD: Cộng $S + (I * I)$
 - ST: Lưu vào S
 - $I := I + 1$ (19–23):
 - LA 0,5: Địa chỉ I
 - LV 0,5: Giá trị I
 - LC 1: Hằng số 1
 - AD: Cộng $I + 1$
 - ST: Lưu vào I
 - J 8: Nhảy về kiểm tra điều kiện
- **Kết thúc (25–28):**
 - LV 0,4: Load giá trị S
 - WRI: In số nguyên (55)
 - WLN: Xuống dòng
 - HL: Dừng chương trình

II. Kết quả thực hiện chương trình với tệp chứa chương trình con (example3.kpl)

1. Mã nguồn file test

```
tests > example3.kpl

1  PROGRAM  EXAMPLE3;  (* TOWER OF HANOI *)
2  VAR      I:INTEGER;
3           N:INTEGER;
4           P:INTEGER;
5           Q:INTEGER;
6           C:CHAR;
7
8  PROCEDURE HANOI(N:INTEGER; S:INTEGER; Z:INTEGER);
9  BEGIN
10     IF N != 0 THEN
11     BEGIN
12         CALL HANOI(N-1,S,6-S-Z);
13         I:=I+1;
14         CALL WRITELN;
15         CALL WRITEI(I);
16         CALL WRITEI(N);
17         CALL WRITEI(S);
18         CALL WRITEI(Z);
19         CALL HANOI(N-1,6-S-Z,Z)
20     END
21 END;  (*END OF HANOI*)
22
23 BEGIN
24     FOR N := 1 TO 4 DO
25     BEGIN
26         FOR I:=1 TO 4 DO
27             CALL WRITEC(' ');
28             C := READC;
29             CALL WRITEC(C)
30         END;
31         P:=1;
32         Q:=2;
33         FOR N:=2 TO 4 DO
34         BEGIN
35             I:=0;
36             CALL HANOI(N,P,Q);
37             CALL WRITELN
38         END
39     END.  (* TOWER OF HANOI *)
```

2. Mô tả chương trình

EXAMPLE3 (Tower of Hanoi) mô phỏng và in các bước giải bài toán Tháp Hà Nội bằng phương pháp đệ quy.

Chương trình chính:

- Nhập và in một số ký tự (minh họa vào/ra I/O).
- Thiết lập cọc nguồn và cọc đích.
- Giải bài toán với số đĩa từ 2 đến 4.

- Với mỗi số đĩa:
 - Khởi tạo lại biến đếm.
 - Gọi procedure HANOI để thực hiện và in các bước di chuyển.

Procedure HANOI:

- Giải bài toán Tháp Hà Nội bằng đệ quy.
- Di chuyển N đĩa từ cọc nguồn sang cọc đích thông qua cọc trung gian.
- Mỗi lần di chuyển:
 - Tăng biến đếm.
 - In thông tin bước (số bước, số đĩa, cọc nguồn, cọc đích).

3. Generated code

```
● → Bai5_Type_Code_Gen ./kplc tests/example3.kpl
0: J 41
1: J 2
2: INT 7
3: HL
4: LC 0
5: NE
6: FJ 41
7: HL
8: LC 1
9: SB
10: HL
11: LC 6
12: HL
13: SB
14: HL
15: SB
16: HL
17: LA 0,4
18: LV 0,4
19: LC 1
20: AD
21: ST
22: WLN
23: LV 0,4
24: WRI
25: HL
26: WRI
27: HL
28: WRI
29: HL
30: WRI
31: HL
32: LC 1
33: SB
34: LC 6
35: HL
36: SB
37: HL
38: SB
39: HL
40: HL
41: INT 9
42: LA 0,5
43: LC 1
44: ST
45: LV 0,5
46: LC 4
47: LE
48: FJ 75
49: LA 0,4
50: LC 1
51: ST
52: LV 0,4
53: LC 4
54: LE
55: FJ 64
56: LC 32
57: WRC
45: LV 0,5
46: LC 4
47: LE
48: FJ 75
49: LA 0,4
50: LC 1
51: ST
52: LV 0,4
53: LC 4
54: LE
55: FJ 64
56: LC 32
57: WRC
63: J 52
64: LA 0,8
65: RC
66: ST
67: LV 0,8
68: WRC
69: LA 0,5
70: LV 0,5
71: LC 1
72: AD
73: ST
74: J 45
75: LA 0,6
76: LC 1
77: ST
78: LA 0,7
79: LC 2
80: ST
81: LA 0,5
82: LC 2
83: ST
84: LV 0,5
85: LC 4
86: LE
87: FJ 102
88: LA 0,4
89: LC 0
90: ST
91: LV 0,5
92: LV 0,6
93: LV 0,7
94: HL
95: WLN
96: LA 0,5
97: LV 0,5
98: LC 1
99: AD
100: ST
101: J 84
102: HL
○ → Bai5_Type_Code_Gen
```

4. Giải thích chi tiết

- PROCEDURE HANOI (Instructions 0–40): Do **CodeGen 1** chưa hỗ trợ đầy đủ việc gọi thủ tục và truyền tham số:
 - Nhiều lệnh quan trọng không được sinh.

- Các đoạn này được thay thế bằng lệnh **HL (halt)**.
- Cấu trúc tổng quát của procedure:
 - *0 (J 41)*: Bỏ qua mã procedure để thực thi chương trình chính trước.
 - *1–2*: Nhảy vào thân procedure và cấp phát stack frame, nhưng chưa sử dụng đúng do thiếu cơ chế gọi thủ tục.
 - *3*: Lẽ ra load tham số **N** từ stack nhưng chưa được cài đặt.
- Phần kiểm tra điều kiện:
 - *4–6*: Kiểm tra **N != 0** để tiếp tục đệ quy.
 - Do chưa load được **N**, phép so sánh không có dữ liệu hợp lệ và có thể thoát procedure ngay.
- Phần xử lý đệ quy:
 - Các bước cần thiết như:
 - Tính cọc trung gian $6 - S - Z$.
 - Gọi đệ quy $HANOI(N-1, S, \text{trung gian})$ và $HANOI(N-1, \text{trung gian}, Z)$.
 - Chưa được CodeGen 1 hỗ trợ nên toàn bộ phần này bị thay bằng **HL**.
- Phần xử lý biến đếm và xuất kết quả:
 - Phép gán **I := I + 1** được sinh mã đúng.
 - Lệnh xuống dòng **WLN** hoạt động chính xác.
 - Các lệnh in thông tin bước (I, N, S, Z) chưa in được giá trị do thiếu load tham số.
- Chương trình chính (Instructions 41–102): Được sinh mã **đầy đủ và chính xác**.
- Khởi tạo stack frame: Cấp phát bộ nhớ cho các biến **I, N, P, Q, C**.
- Các vòng lặp chính:
 - Vòng lặp **N := 1 TO 4**: Minh họa vào/ra bằng cách in ký tự và đọc ký tự từ bàn phím.
 - Vòng lặp **N := 2 TO 4**:
 - Khởi tạo lại biến đếm **I**.
 - Đáng lẽ gọi $HANOI(N, P, Q)$ để giải bài toán, nhưng chưa được implement do hạn chế của CodeGen 1.
 - In dòng mới sau mỗi lần lặp.

